

GazeComp – Gaze-Driven Dynamic Multi-Band Compressor

Development Report

Name/Student ID: ZiCheng Yin / 11231126

Project Type: Audio Plugin (VST3) + standalone exe

Development Platform: JUCE Framework / C++ / CMake

1. Motivation

The core concept of multi-band compressors – “crossover points” – relies entirely on auditory experience to understand, creating a steep learning curve for beginners. At the same time, DJs often find themselves “overwhelmed” during live performances, where gaze control can free up their hands. In China, there are over 85 million people with disabilities, and individuals with physical impairments face significant barriers when operating traditional audio software.

Based on these observations, I developed two key ideas: making crossover points “visualizable” and audio control “contactless”. With these goals in mind, I began developing GazeComp.

2. Technical Architecture

Audio Processing



I selected JUCE as the development framework, implementing the core algorithms within the `processBlock()` function. The filter uses a Linkwitz-Riley structure to ensure phase alignment at crossover points,

preventing audio coloration. The input audio is separated into low-frequency band (20–200Hz), mid-frequency band (200–2000Hz), and high-frequency band (2000–20000Hz), controlled by freqLow and freqMid.

After separating into three bands, each band undergoes independent compression. The compressor core is an envelope follower – detecting signal amplitude, smoothing tracking through Attack/Release, comparing with Threshold, calculating gain reduction based on Ratio, and using Makeup Gain to compensate for level loss. All parameters are exposed to the host via juce::AudioParameterFloat, supporting automation recording.

Gaze Signal Bridging

I chose the Tobii Eye Tracker 4C as my develop tool. The Tobii 4C is a consumer-grade eye tracker, but its advanced SDK requires commercial licensing, making raw gaze data inaccessible to 普通 users. I experimented with Python's `tobii-research` (requiring discontinued Pro upgrade keys), C# calls to `Tobii.StreamEngine.Net.dll` (with data streams locked by permissions), and ultimately used the community-developed GazeOSC, which reads Tobii 4C gaze data through the legacy EyeX API and converts it to OSC protocol output.

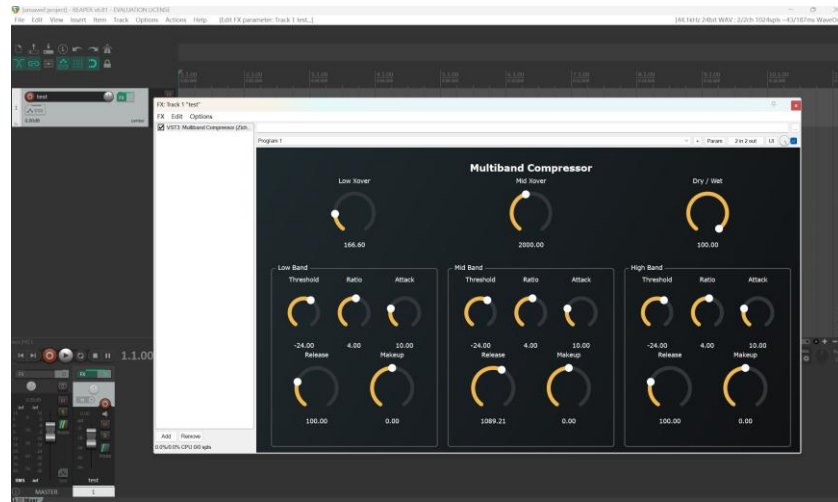
Workflow: GazeOSC reads Tobii 4C gaze coordinates → UDP transmits OSC → JUCE's OSCReceiver receives → x-coordinate maps to freqLow ($x \approx 0$ corresponds to 80Hz crossover, $x \approx 1$ corresponds to 500Hz). Gaze coordinates are smoothed with filtering to prevent parameter jitter and unnatural listening experience.



Engineering Setup

CMake is used for build management, depending on JUCE 8.x modules. Configured for both VST3 and Standalone compilation targets, compiled

using VS Code's CMake Tools extension. The generated .vst3 file is automatically installed to the system's VST3 directory.



3. Development Process

Audio Component: The first successful compilation produced no compression effect. After troubleshooting, I discovered the gain reduction calculation in the envelope follower had the wrong sign. After correction, the compression function worked properly.

Gaze Component: This was the most time-consuming part. After three failed attempts with different approaches, I finally succeeded with GazeOSC outputting gaze coordinates, completing the entire pipeline.

Integration and Debugging: Raw gaze coordinates caused jitter in crossover points. After adding first-order low-pass filtering for smoothing, gaze movement and sound changes became synchronized.

```
private:
{
#include <juce_audio_processors/juce_audio_processors.h>
#include <juce_audio_devices/juce_audio_devices.h>
#include <juce_audio_formats/juce_audio_formats.h>
#include <juce_dsp/juce_dsp.h>

#include <array>
#include <atomic>

class PluginProcessor : public juce::AudioProcessor
{
public:
    PluginProcessor();
    ~PluginProcessor() override;

    void prepareToPlay(double sampleRate, int samplesPerBlock) override;
    void releaseResources() override;

    bool isBusesLayoutSupported(const BusesLayout& layouts) const override;

    void processBlock(juce::AudioBuffer<float>&, juce::MidiBuffer&) override;

    juce::AudioProcessorEditor* createEditor() override;
    bool hasEditor() const override;

    const juce::String getName() const override;

    bool acceptsMidi() const override;
    bool producesMidi() const override;
    bool isMidiEffect() const override;
    double getTailLengthSeconds() const override;

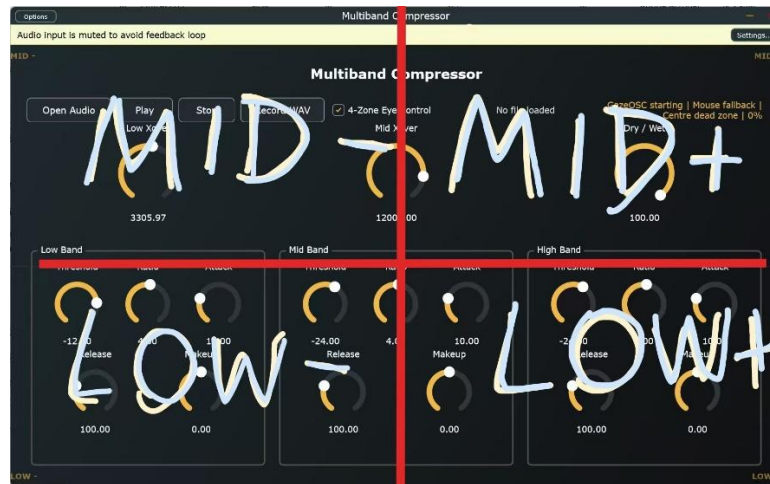
    int getNumPrograms() override;
    int getCurrentProgram() override;
    void setCurrentProgram(int index) override;
    const juce::String getProgramName(int index) override;
```

4. Interaction Design

Traditional compressor interfaces are cluttered with knobs and

numerical displays, making it difficult for beginners to establish the connection between operations and auditory perception.

My approach transforms "parameter adjustment" into "spatial positioning": when the user looks left, the crossover point lowers, narrowing the low-frequency compression range; when looking right, the crossover point raises, expanding the compression coverage across a wider frequency range.

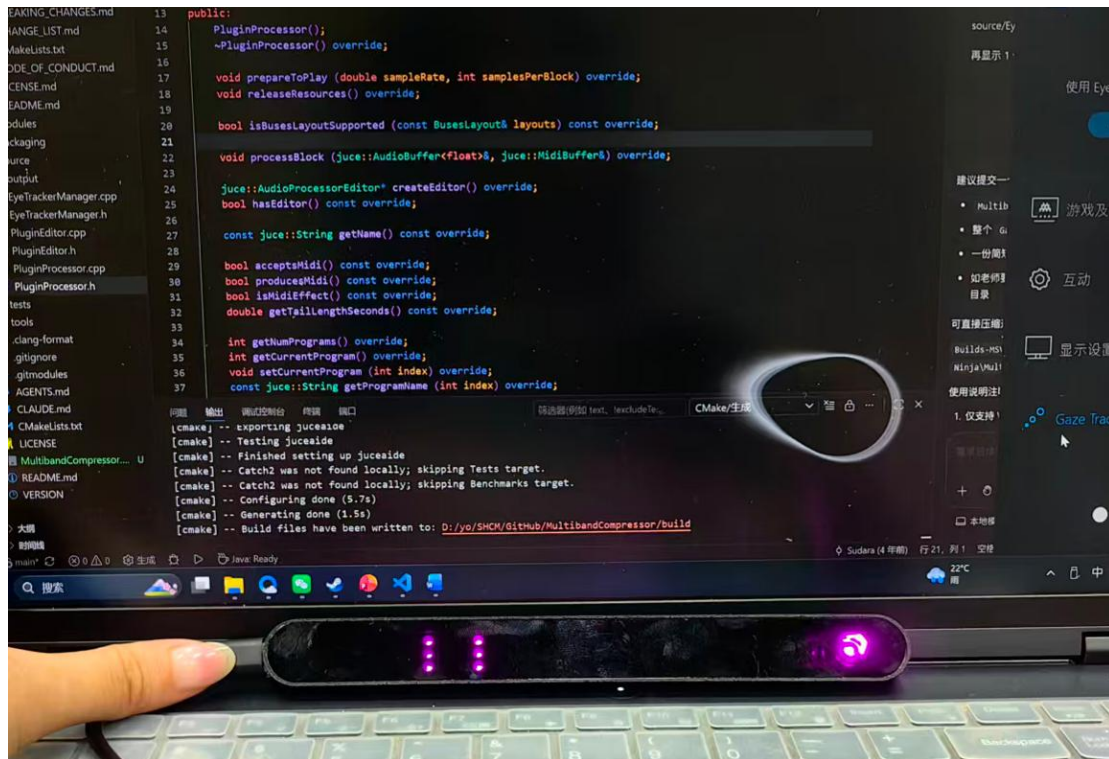


This creates an immediate "gaze position - crossover point value - auditory result" feedback loop, allowing beginners to understand crossover points within minutes, eliminating the need for 反复 trial-and-error listening.

5. Results and Future Work

Achieved Features:

- Linkwitz-Riley three-band crossover with real-time adjustable crossover points
- Independent control of Threshold, Ratio, Attack, Release, and Makeup Gain for each band
- The plugin loads stably in DAWs like Reaper, and the standalone exe runs smoothly with basic features such as audio loading and playback, keeping CPU usage under 5%.



Future Plans:

Introduce gaze dwell time recognition for multi-parameter control; add teaching mode to display corresponding frequency ranges in real-time; extend gaze control to EQ, reverb, and other effects, providing contactless mixing solutions for individuals with physical disabilities.

Conclusion

The significance of audio tools goes beyond "making sound better" – it's about "making sound more accessible to understand". When technology builds a bridge between perception and cognition, it transcends being merely a tool and becomes a medium for creating possibilities.